USGS Stream Discharge Analysis

A Reproducible Hydrology Workflow in R

marc

April 24, 2026

Contents

---

1 Before You Begin: Getting Set Up

Work through **every step in this section once** before touching any code. You only need to do this setup once per computer.

1.1 Step 1 — Open RStudio

1.1.1 Step 2 Lots of step we don't need...

1.2 Step 3 — Install Required R Packages

In the RStudio **Console**, run the line below. Copy and paste the whole thing at once:

```
install.packages(c("dataRetrieval", "tidyverse", "lubridate",
                   "scales", "knitr"))
```

Wait for the `>` prompt to reappear before continuing. If asked *"Do you want to install from sources?"*, type `n` and press Enter.

1.3 Step 4 — Create a New R Markdown File

1. In RStudio, go to **File** → **New File** → **R Markdown...**
2. In the dialog box that opens:
   - Title: type anything (e.g. *USGS Discharge*)
   - Author: your name
   - Default Output Format: select **PDF**
   - Click **OK**
3. RStudio creates a template file. **Delete everything** from line 6 downward (keep only the YAML header block at the top, the six lines between the `---` markers).
4. Paste the contents of this document in below the YAML header.
5. Save the file with **File** → **Save** (`.Rmd` extension is added automatically).

1.4 Step 5 — Knit to PDF

Click the **Knit** button at the top of the editor pane (or press `Ctrl+Shift+K` on Windows/Linux, `Cmd+Shift+K` on Mac).

A PDF will appear in the same folder as your `.Rmd` file. If you get an error, read the red message in the console — it almost always tells you exactly which package is missing or which line has a problem.

---

2 How This Document Works

2.1 What is R Markdown?

R Markdown (`.Rmd`) is a plain-text file that weaves together two things:

- **Prose** written in Markdown (the text you are reading right now).
- **Code chunks** — blocks of R code surrounded by triple backticks that run automatically when you knit.

When you click **Knit**, RStudio runs every code chunk from top to bottom, collects the output (numbers, tables, plots), and assembles everything into a single PDF. This means the results in the PDF are *always* produced by the code you can see — nothing is copy-pasted by hand.

2.2 Document Structure

Section	Purpose
Settings	Change site number and dates here only
Data Acquisition	Downloads data from USGS automatically
Data Cleaning	Removes missing values, adds helper columns
Summary Statistics	Annual table
Plots	Four standard hydrological visualisations
Your Assignment	Instructions for finding your own station

---

3 Required Packages

Each package is loaded here at the top so they are available throughout the entire document. You installed these in Step 3 above; this `library()` call simply loads them into the current session.

```r
library(dataRetrieval)  # official USGS R package - fetches NWIS data
library(tidyverse)      # ggplot2 (plots) + dplyr/tidyr (data wrangling)
library(lubridate)      # easy date/time arithmetic
library(scales)         # comma-formatted and log axis labels in ggplot2
library(knitr)          # kable() for clean PDF tables
```

---

4 Settings — The Only Lines You Need to Change

Every user-defined parameter lives in this one chunk. To analyse a different river, you change **only these lines** and re-knit. You do not need to touch anything else in the document.

```r
# ---------------------------------------------------------------
# PART 1: Run this document exactly as-is using these defaults.
#         The Potomac River example should produce a complete PDF
#         with no errors. Confirm that before changing anything.
# ---------------------------------------------------------------

site_no  <- "01646500"   # Potomac River at Little Falls, MD
param_cd <- "00060"      # 00060 = discharge (cfs)
                         # 00065 = gage height (ft)
stat_cd  <- "00003"      # 00003 = daily mean
start_dt <- "2022-01-01"
end_dt   <- "2023-12-31"

# ---------------------------------------------------------------
# PART 2 (assignment): Replace site_no, start_dt, and end_dt
#         with values for a station in your state.
#         See Section 8 for instructions on finding a station.
# ---------------------------------------------------------------
```

---

5 Data Acquisition

5.1 Download Directly in R *(recommended)*

`readNWISdv()` sends a query to the USGS National Water Information System (NWIS) daily-values API and returns a tidy data frame directly into R — no browser, no CSV, no copy-pasting. `renameNWISColumns()` replaces the cryptic auto-generated column name (e.g. `X_00060_00003`) with the readable name `Flow`. `readNWISsite()` pulls station metadata (official name, coordinates, drainage area) so plot titles are generated automatically.

```r
raw  <- readNWISdv(
  siteNumbers = site_no,
  parameterCd = param_cd,
  startDate   = start_dt,
  endDate     = end_dt
)

raw  <- renameNWISColumns(raw)
meta <- readNWISsite(site_no)

glimpse(raw)
```

```
## Rows: 730
## Columns: 5
## $ agency_cd <chr> "USGS", "USGS", "USGS", "USGS", "USGS", "USGS", "USGS", "USG~
## $ site_no   <chr> "01646500", "01646500", "01646500", "01646500", "01646500", ~
```

```
## $ Date      <date> 2022-01-01, 2022-01-02, 2022-01-03, 2022-01-04, 2022-01-05,~
## $ Flow      <dbl> 3370, 5320, 7380, 13800, 13600, 12000, 10800, 9440, 8130, 79~
## $ Flow_cd   <chr> "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", ~
```

NOTE: Sometimes this doesn't work – I don't kno why! Let me know if you have trouble here!

5.2 Alternative: Load a Pre-Downloaded CSV

If your computer cannot reach the internet, you can download a CSV manually from https://waterdata.usgs.gov/nwis and load it here. Set `eval=TRUE` on this chunk and `eval=FALSE` on the chunk above.

The expected column headers from an NWIS CSV export are: `x, y, id, time_series_id, monitoring_location_id, parameter_code, statistic_id, time, value, unit_of_measure, approval_status, qualifier, last_modified`

```
# raw_csv <- read_csv("your_file.csv") %>%
#   mutate(time = ymd_hms(time)) %>%
#   filter(parameter_code == "00060") %>%
#   rename(Date = time, Flow = value) %>%
#   filter(approval_status == "A")
#
# raw <- raw_csv %>% select(Date, Flow)
```

---

6 Data Cleaning

This chunk does four things:

1. **Selects** only `Date` and `Flow` — all other NWIS columns are dropped.
2. **Removes** rows where `Flow` is `NA` (equipment malfunctions and provisional data gaps appear as `NA`).
3. **Derives** three helper columns: `year`, `month` (Jan–Dec factor), and `doy` (day of year, 1–366).
4. **Creates** two text labels used in every plot title and y-axis so they update automatically when you switch stations.

```
df <- raw %>%
  select(Date, Flow) %>%
  filter(!is.na(Flow)) %>%
  mutate(
    year  = year(Date),
    month = month(Date, label = TRUE),
    doy   = yday(Date)
  )

site_name <- meta$station_nm
units_lab  <- "Discharge (cfs)"
```

```r
cat("Records loaded :", nrow(df), "\n")
```

```
## Records loaded : 730
```

```r
cat("Date range     :", as.character(min(df$Date)),
    "to", as.character(max(df$Date)), "\n")
```

```
## Date range     : 2022-01-01 to 2023-12-31
```

```r
cat("Site           :", site_name, "\n")
```

```
## Site           : POTOMAC RIVER NEAR WASH, DC LITTLE FALLS PUMP STA
```

---

7 Summary Statistics

`dplyr::summarise()` groups the data by calendar year and computes six statistics for each year. `knitr::kable()` formats the resulting data frame as a clean LaTeX table. The `booktabs = TRUE` option uses horizontal rules instead of a full grid, which is the standard style in scientific journals.

```r
stats <- df %>%
  group_by(year) %>%
  summarise(
    N      = n(),
    Mean   = round(mean(Flow),   1),
    Median = round(median(Flow), 1),
    SD     = round(sd(Flow),     1),
    Min    = round(min(Flow),    1),
    Max    = round(max(Flow),    1),
    .groups = "drop"
  )

kable(
  stats,
  booktabs    = TRUE,
  caption     = paste("Annual discharge summary \u2014", site_name),
  col.names   = c("Year", "N", "Mean", "Median", "SD", "Min", "Max"),
  format.args = list(big.mark = ",")
)
```

Table 2: Annual discharge summary — POTOMAC RIVER NEAR WASH, DC LITTLE FALLS PUMP STA

Year	N	Mean	Median	SD	Min	Max
2,022	365	8,906.4	6,200	10,343.2	1,750	102,000
2,023	365	5,710.4	3,990	5,383.3	450	35,700

8 Plots

8.1 Daily Time Series

A linear y-axis gives the most intuitive first view of the record. Large flood peaks stand out clearly. `scale_x_date()` controls how date tick marks are spaced and formatted along the x-axis. `comma` from the `scales` package prevents R from writing large numbers in scientific notation (e.g. `1e+05`).

```
ggplot(df, aes(x = Date, y = Flow)) +
  geom_line(color = "steelblue", linewidth = 0.5) +
  scale_x_date(date_labels = "%b %Y", date_breaks = "3 months") +
  scale_y_continuous(labels = comma) +
  labs(
    title    = paste("Daily Mean Discharge \u2014", site_name),
    subtitle = paste("USGS site", site_no),
    x        = NULL,
    y        = units_lab
  ) +
  theme_bw(base_size = 11) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

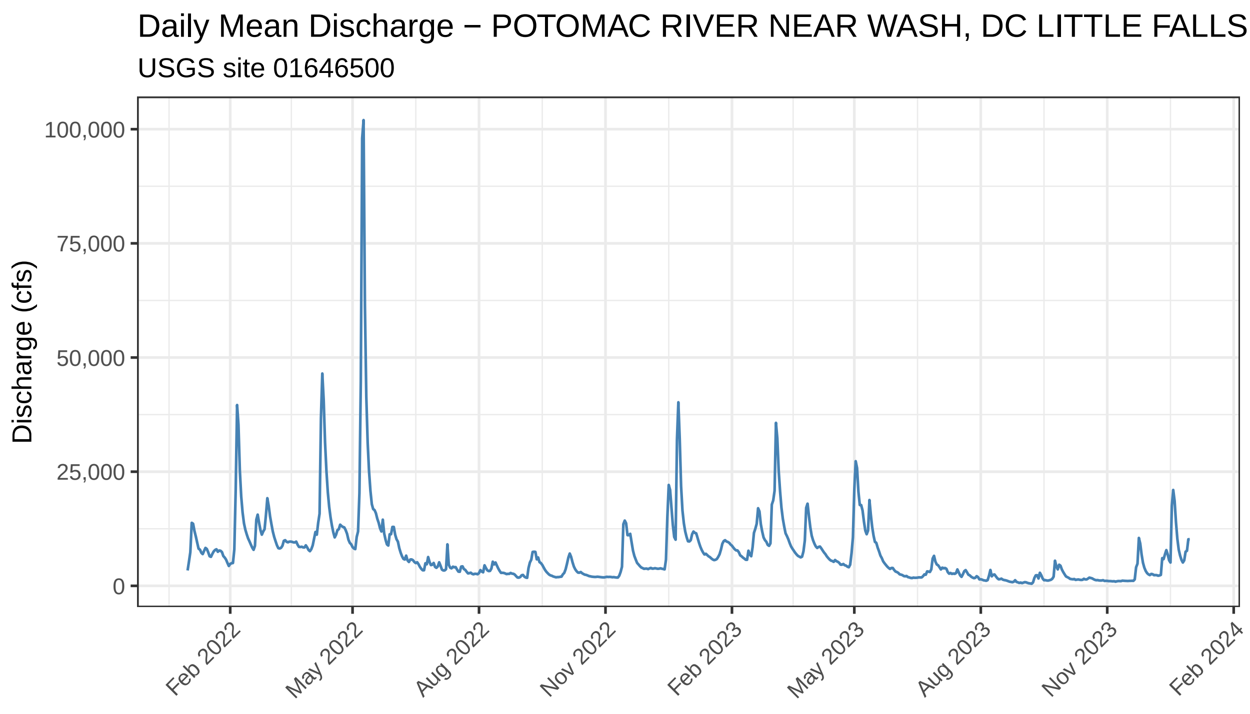

Figure 1: Daily mean discharge over the full period of record.

8.2 Log-Scale Time Series

Stream discharge is highly skewed: flood peaks can be 100–1,000 times larger than low flows. A $\log_{10}$ y-axis compresses the peaks and stretches the low-flow period so both are readable at the same time. `annotation_logticks()` adds the characteristic minor tick marks seen on logarithmic axes in textbooks.

```
ggplot(df, aes(x = Date, y = Flow)) +
  geom_line(color = "steelblue", linewidth = 0.5) +
  scale_x_date(date_labels = "%b %Y", date_breaks = "3 months") +
  scale_y_log10(labels = comma) +
  annotation_logticks(sides = "l") +
  labs(
    title    = paste("Daily Mean Discharge (log scale) \u2014", site_name),
    subtitle = paste("USGS site", site_no),
    x        = NULL,
    y        = paste0("log\u2081\u2080 ", units_lab)
  ) +
  theme_bw(base_size = 11) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

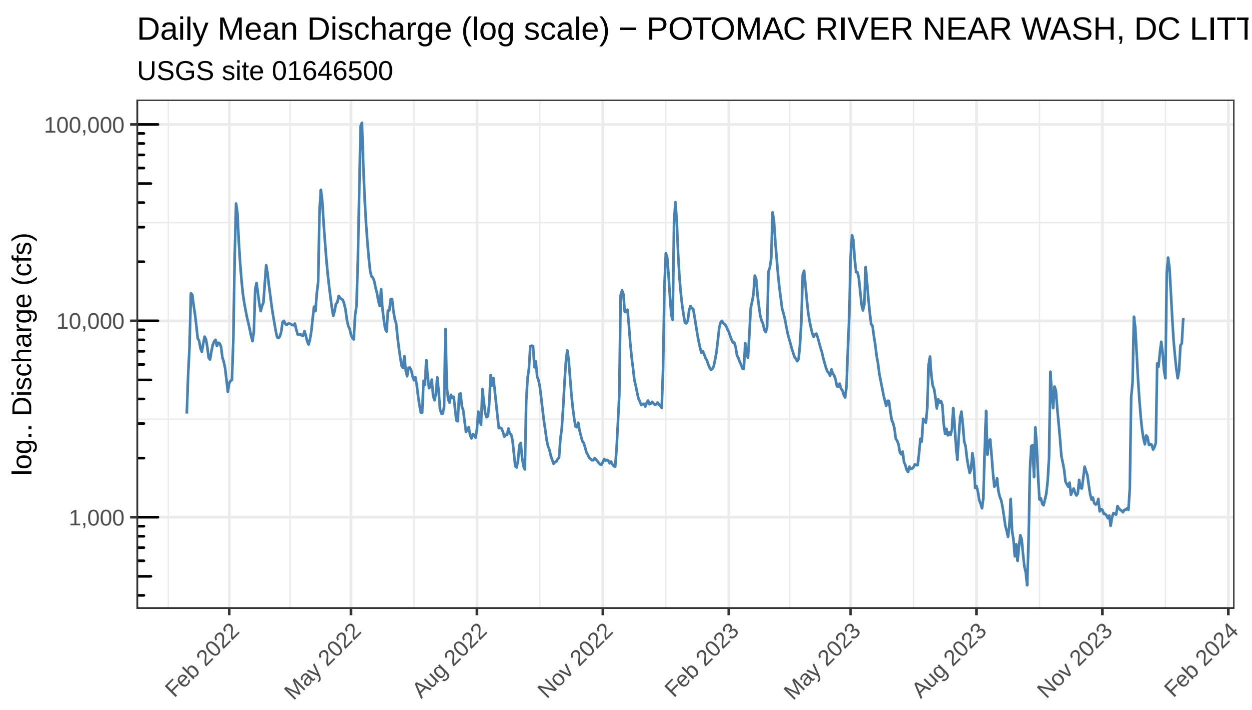

Figure 2: Daily mean discharge on a log-10 y-axis.

8.3 Monthly Box Plots

Grouping by calendar month and pooling all years reveals the **seasonal pattern** of discharge independent of year-to-year variability. The box shows the interquartile range (25th–75th percentile), the horizontal line inside the box is the median, and points beyond the whiskers are outliers. A log y-axis is kept here because within-month distributions are also skewed.

```
ggplot(df, aes(x = month, y = Flow, fill = month)) +
  geom_boxplot(outlier.size  = 0.8,
               outlier.alpha = 0.4,
               show.legend   = FALSE) +
  scale_y_log10(labels = comma) +
  scale_fill_brewer(palette = "Paired") +
  annotation_logticks(sides = "l") +
  labs(
    title = paste("Monthly Discharge Distribution \u2014", site_name),
    x     = NULL,
    y     = paste0("log\u2081\u2080 ", units_lab)
  ) +
  theme_bw(base_size = 11)
```

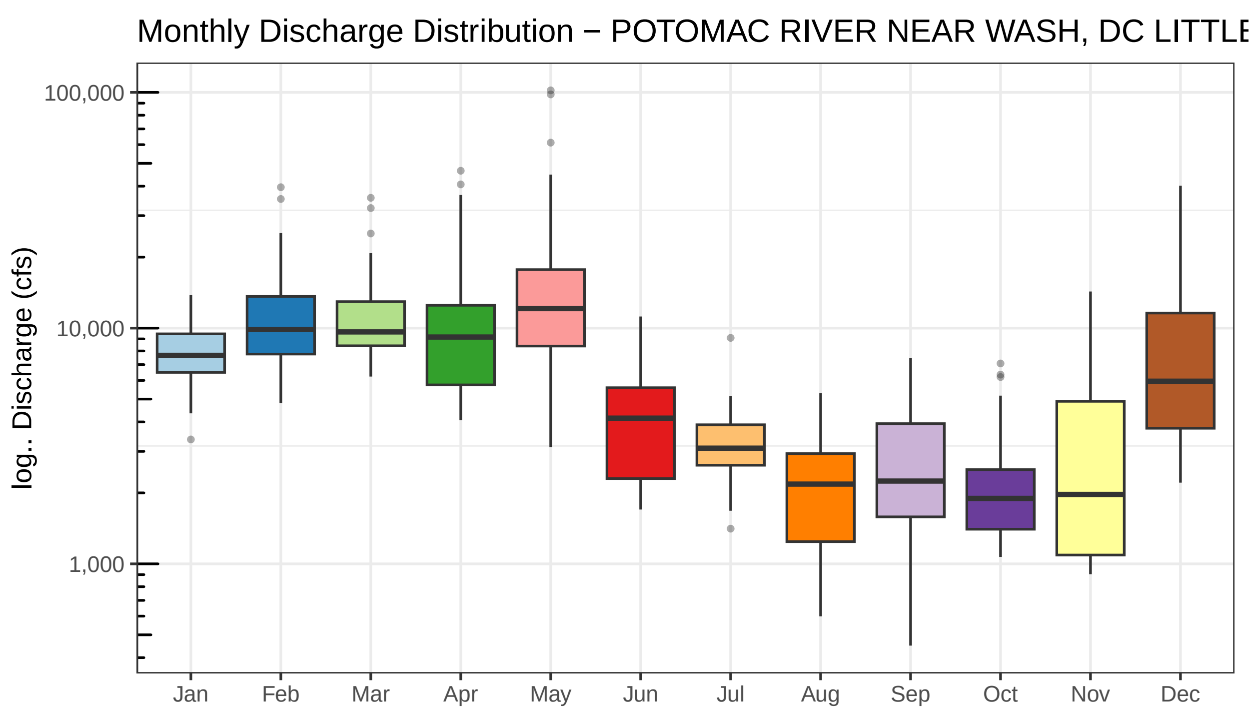

Figure 3: Distribution of daily discharge by calendar month, all years pooled.

8.4 Flow Duration Curve

The **flow duration curve (FDC)** is one of the most widely used tools in hydrology. It answers the question: *"What discharge is equalled or exceeded X% of the time?"* The curve is constructed by ranking all daily values from highest to lowest and computing each value's exceedance percentage. A **steep FDC** indicates a flashy, rainfall-driven stream with large contrasts between floods and droughts. A **flat FDC** suggests a stream sustained by groundwater baseflow with relatively stable discharge.

```r
fdc <- df %>%
  arrange(desc(Flow)) %>%
  mutate(exceed_pct = 100 * row_number() / n())

ggplot(fdc, aes(x = exceed_pct, y = Flow)) +
  geom_line(color = "steelblue", linewidth = 0.7) +
  scale_y_log10(labels = comma) +
  scale_x_continuous(
    breaks = seq(0, 100, 10),
    labels = paste0(seq(0, 100, 10), "%")
  ) +
  annotation_logticks(sides = "l") +
  labs(
    title = paste("Flow Duration Curve \u2014", site_name),
    x     = "Percent of time flow equalled or exceeded",
    y     = paste0("log\u2081\u2080 ", units_lab)
  ) +
  theme_bw(base_size = 11)
```

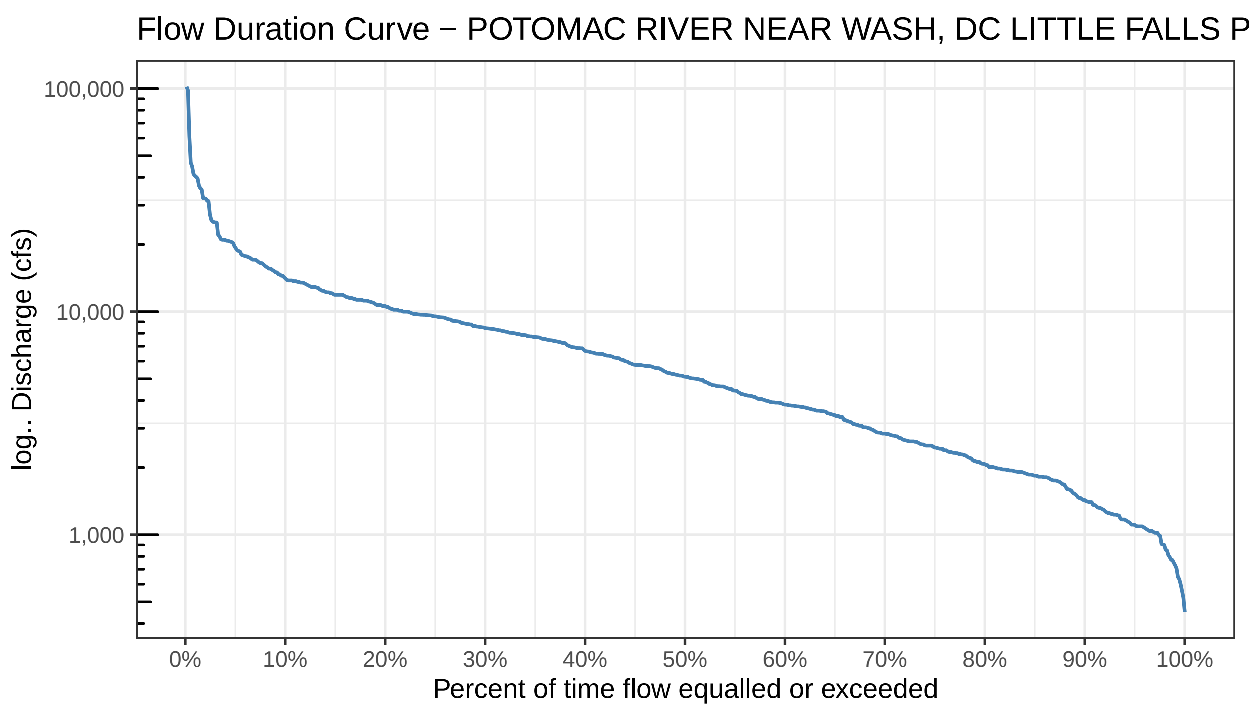

Figure 4: Flow duration curve for the full period of record.

9 Your Assignment: Find a Station in Your State

9.1 Step 1 — Confirm the Potomac Example Works

Before changing anything, knit this document exactly as provided and confirm you get a complete PDF with four plots and a table. The Potomac River site (`01646500`) has a long, continuous record and is a reliable test case. If the Potomac run fails, fix that error before moving on.

9.2 Step 2 — Find a USGS Station in Your State

Go to the **USGS National Map of Current Water Conditions**:

> https://waterwatch.usgs.gov

Or use the **NWIS Mapper**:

> https://maps.waterdata.usgs.gov/mapper/index.html

1. Zoom to your state.
2. Click on a stream gauge dot — a popup appears with the site number (an 8-digit code like `03015500`), site name, and current status.
3. Choose a site that shows **streamflow (discharge)** data, not just stage.
4. Click **Access Data** in the popup and confirm the site has at least two full years of approved daily discharge records.

Alternatively, search by state name using `dataRetrieval` directly in the RStudio console:

```r
# Run this in the Console to browse sites in your state.
# Replace "VA" with your two-letter state abbreviation.
my_sites <- whatNWISsites(
  stateCd     = "VA",
  parameterCd = "00060",
  service     = "dv"
)
View(my_sites)   # opens a spreadsheet-style viewer in RStudio
```

9.3 Step 3 — Update the Settings Chunk

Once you have a site number, return to **Section 4 – Settings** and replace the three values:

```r
site_no  <- "YOUR_8_DIGIT_SITE_NUMBER"
start_dt <- "YYYY-MM-DD"   # at least 2 full years recommended
end_dt   <- "YYYY-MM-DD"
```

Everything else in the document updates automatically.

9.4 Step 4 — Re-Knit and Interpret

Click **Knit → Knit to PDF** again. In your write-up, address the following questions (add a new section below this one in your `.Rmd`):

1. What river and state did you choose, and why?
2. What are the peak discharge months at your site? What drives that seasonality (snowmelt, hurricane season, spring rains, etc.)?
3. Compare the shape of your flow duration curve to the Potomac's. Is your stream more or less flashy? What might explain the difference?
4. Are there any data gaps or `NA` values in your record? What might cause missing data at a USGS gauge?

---

10 Session Info

Recording session information at the end of every report is good scientific practice. Anyone trying to reproduce your results can see exactly which versions of R and each package were used.

```r
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: x86_64-pc-linux-gnu
## Running under: Rocky Linux 8.10 (Green Obsidian)
##
## Matrix products: default
## BLAS/LAPACK: /usr/lib64/libopenblas-r0.3.15.so;  LAPACK version 3.9.0
##
## locale:
##  [1] LC_CTYPE=en_US.UTF-8       LC_NUMERIC=C
##  [3] LC_TIME=en_US.UTF-8        LC_COLLATE=en_US.UTF-8
##  [5] LC_MONETARY=en_US.UTF-8    LC_MESSAGES=en_US.UTF-8
##  [7] LC_PAPER=en_US.UTF-8       LC_NAME=C
##  [9] LC_ADDRESS=C               LC_TELEPHONE=C
## [11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C
##
## time zone: America/Los_Angeles
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] knitr_1.51          scales_1.4.0        lubridate_1.9.4
##  [4] forcats_1.0.1       stringr_1.6.0       dplyr_1.1.4
##  [7] purrr_1.2.1         readr_2.1.6         tidyr_1.3.2
## [10] tibble_3.3.1        ggplot2_4.0.1       tidyverse_2.0.0
```

```
## [13] dataRetrieval_2.7.24
##
## loaded via a namespace (and not attached):
##  [1] rappdirs_0.3.4      generics_0.1.4      xml2_1.5.2
##  [4] class_7.3-23        KernSmooth_2.23-26  stringi_1.8.7
##  [7] hms_1.1.4           digest_0.6.39       magrittr_2.0.4
## [10] timechange_0.3.0    evaluate_1.0.5      grid_4.5.2
## [13] RColorBrewer_1.1-3  fastmap_1.2.0       e1071_1.7-17
## [16] DBI_1.2.3           tinytex_0.59        httr2_1.2.2
## [19] cli_3.6.5           crayon_1.5.3        rlang_1.1.7
## [22] units_1.0-0         bit64_4.6.0-1       withr_3.0.2
## [25] yaml_2.3.12         otel_0.2.0          parallel_4.5.2
## [28] tools_4.5.2         tzdb_0.5.0          curl_7.0.0
## [31] vctrs_0.7.1         R6_2.6.1            proxy_0.4-29
## [34] lifecycle_1.0.5     classInt_0.4-11     bit_4.6.0
## [37] vroom_1.7.0         pkgconfig_2.0.3     pillar_1.11.1
## [40] gtable_0.3.6        data.table_1.18.2.1 glue_1.8.0
## [43] Rcpp_1.1.1          sf_1.0-24           xfun_0.56
## [46] tidyselect_1.2.1    rstudioapi_0.18.0   farver_2.1.2
## [49] htmltools_0.5.9     labeling_0.4.3      rmarkdown_2.30
## [52] compiler_4.5.2      S7_0.2.1
```